FADENA | Universidad de la Defensa Nacional

SQL Avanzado

Temario

- **Sentencias de Manipulación de Datos (DML):**
  - Subconsultas (consultas anidadas): comparación, IN, NOT IN, ANY, ALL.
  - Operadores de Conjunto: UNION, INTERSECT, EXCEPT.

- **Sentencias de Definición de Datos (DDL):** Vistas, Índices, otros.

- **Sentencias de Control de Datos (DCL).**

- **Sentencias de Control de Transacciones (TCL).**

Subconsultas (DML)

Una **subconsulta**, o **consulta anidada**, es una sentencia **SELECT dentro de otra sentencia SQL**.

- Pueden usarse dentro de SELECT, INSERT, UPDATE o DELETE.
- Se encierra entre paréntesis.
- Pueden anidarse unas dentro de otras.
- **Consulta externa o principal**: aquella que incluye a la subconsulta.

Pueden retornar: **un único valor (escalar), una única fila, múltiples filas**. Según sea el caso, se pueden encontrar en diferentes cláusulas:

- WHERE: filtrar registros.
- HAVING: filtrar grupos.
- FROM: como tabla temporal.
- SELECT: incluir un valor calculado.

Subconsultas en cláusulas WHERE y HAVING (DML)

El predicado en estas cláusulas está formado por **comparaciones** unidas por AND/OR.

Cada comparación involucra dos **operandos** que son: columnas, constantes (o expresiones que contengan ambas), y ahora veremos también **resultados de subconsultas**.

- **Operadores** que se pueden utilizar con las **subconsultas**:
  - Operadores básicos de comparación (=, <>, >, <, >=, <=).
  - IN, NOT IN.
  - ANY, ALL.
  - EXISTS y NOT EXISTS.

Subconsultas en cláusulas WHERE y HAVING (DML)

Operadores Básicos de Comparación (=, <>, >, <, >=, <=)

expresion operador (subconsulta) => La subconsulta debe devolver **un solo valor**.

El predicado será verdadero si la comparación entre la expresión y el resultado de la subconsulta es verdadera.

```
SELECT *
  FROM producto
WHERE precio >= (SELECT MAX(precio)
                 FROM producto P INNER JOIN fabricante F
                   ON P.codigo_fabricante  = F.codigo
                 WHERE fabricante.nombre = 'Samsung');
```

(*) Productos con precio mayor o igual al más caro de Samsung

```
SELECT departamentoID, AVG(sueldo)
  FROM empleados
GROUP BY departamentoID
HAVING AVG(sueldo) > ( SELECT AVG(sueldo)
                       FROM empleados );
```

(*) Departamentos con sueldo promedio mayor al promedio global

Subconsultas en cláusulas WHERE y HAVING (DML)

Operadores IN y NOT IN

expresion [NOT] IN (subconsulta) => La subconsulta puede devolver **varias filas**.

IN: verdadero si la expresión es **igual a alguno** de los valores devueltos por la subconsulta.

```
SELECT ciudadID, nombre
  FROM ciudad
WHERE ciudadID IN ( SELECT ciudadID FROM clientes);
```

(*) Ciudades con al menos un cliente

NOT IN: verdadero si la expresión es **distinta a todos** los valores devueltos por la subconsulta, **o si esta no devuelve ninguna fila**.

```
SELECT *
  FROM cliente
WHERE clienteID NOT IN (SELECT clienteID FROM pedido);
```

(*) Clientes sin ningún pedido

Subconsultas en cláusulas WHERE y HAVING (DML)

Operadores ANY y ALL : Se utilizan con operadores de comparación (=, <>, >, <, >=, <=)

expresion operador [ANY | ALL] (subconsulta) => La subconsulta puede devolver **varias filas**.

ANY: verdadero si la comparación es verdadera **para al menos uno** de los valores devueltos por la subconsulta.

```
SELECT empleadoID, nombre
  FROM empleados
WHERE categoria = 'operario'
    AND sueldo > ANY (SELECT sueldo
                        FROM empleados
                       WHERE categoria = 'supervisor');
```

(*) Operarios que ganan más que ALGÚN supervisor

ALL: verdadero si la comparación es verdadera **para todos** los valores devueltas por la subconsulta, **o si esta no devuelve ninguna fila**.

```
SELECT empleadoID, nombre
  FROM empleados
WHERE categoria = 'operario'
    AND sueldo > ALL (SELECT sueldo
                        FROM empleados
                       WHERE categoria = 'supervisor');
```

(*) Operarios que ganan más que TODOS los supervisores

Subconsultas en cláusulas WHERE y HAVING (DML)

Operadores ANY y ALL : Cuando no están disponibles (por ej. en SQLite), hay que reescribir la consulta usando **MIN/MAX** (para comparaciones con >, <, >=, <=) o **IN/NOT IN** (para = y <>)

Las equivalencias son:

expr = ANY (subconsulta) => expr IN (subconsulta)

expr <> ALL (subconsulta) => expr NOT IN (subconsulta)

expr > ANY (subconsulta) => expr > (SELECT MIN(...) FROM ...)

expr > ALL (subconsulta) => expr > (SELECT MAX(...) FROM ...)

expr < ANY (subconsulta) => expr < (SELECT MAX(...) FROM ...)

expr < ALL (subconsulta) => expr < (SELECT MIN(...) FROM ...)

```
SELECT empleadoID, nombre
  FROM empleados
WHERE categoria = 'operario'
    AND sueldo > ALL (SELECT sueldo FROM empleados
                      WHERE categoria = 'supervisor');
```

```
SELECT empleadoID, nombre
  FROM empleados
WHERE categoria = 'operario'
    AND sueldo > (SELECT MAX(sueldo) FROM empleados
                  WHERE categoria = 'supervisor');
```

Subconsultas en cláusulas WHERE y HAVING (DML)

Sobre los operadores de subconsultas vistos (operadores de comparación, IN, NOT IN, ANY, ALL)

Se pueden usar también comparando **más de un valor**:

(expr1, expr2, …) operador (subconsulta)

(expr1, expr2, …) [NOT] IN (subconsulta)

(expr1, expr2, …) operador [ANY | ALL] (subconsulta)

En estos casos, **la subconsulta debe devolver tantas columnas como las especificadas entre paréntesis a la izquierda**. Las comparaciones, se harán por cada campo en las filas de ambos lados.

```
SELECT *
  FROM facturas
WHERE (descuento, monto) = ( SELECT MAX(descuento),
                                    MAX(monto)
                             FROM facturas );
```

(*) Facturas con descuento y monto máximo

Subconsultas en cláusulas WHERE y HAVING (DML)

Sobre los operadores de subconsultas vistos (NOT IN y ALL)

En estos casos en particular, **si la subconsulta devuelve algún nulo**, el predicado se evaluará como nulo (más allá de que existan otras filas con valor), y **la consulta externa no devolverá registros**.

La solución es incluir en la subconsulta una condición de **"IS NOT NULL"** para los campos a devolver, o usar funciones como **COALESCE**.

```
SELECT *
FROM facturas
WHERE descuento >= ALL ( SELECT COALESCE(descuento, 0)
                         FROM facturas);
```

(*) Si pudieran existir facturas con descuento NULL.
Alternativamente se podría usar "WHERE descuento IS NOT NULL"

Subconsultas en cláusulas WHERE y HAVING (DML)

Operadores EXISTS y NOT EXISTS

[EXISTS | NOT EXISTS] (subconsulta) => La subconsulta referencia a columnas de la consulta externa.
No evalúa valores, sólo pregunta "¿devolvió algo?"

EXISTS: verdadero si la subconsulta **devuelve al menos una fila.**

```
SELECT clienteID, nombre FROM cliente C
WHERE EXISTS ( SELECT pedidoID
               FROM pedido P
               WHERE C.clienteID = P.clienteID
                 AND P.fecha > '2025-08-10');
```

(*) Clientes con pedido posterior al 10/08/2025

NOT EXISTS: verdadero si la subconsulta **no devuelve ninguna fila**.

```
SELECT * FROM cliente C
WHERE NOT EXISTS (SELECT pedidoID
                  FROM pedido P
                  WHERE C.clienteID = P. clienteID);
```

(*) Clientes sin ningún pedido

Subconsultas Correlacionadas y No Correlacionadas

NO CORRELACIONADA

La subconsulta **se ejecuta UNA sola vez, independientemente de la consulta externa**, sin hacer referencias a sus datos. Su resultado se utiliza como entrada fija para la consulta externa.

Orden: ① subconsulta → ② consulta externa

```
SELECT nombre, precio
  FROM productos
WHERE precio > ( SELECT AVG(precio)
                 FROM productos);
```

(*) Productos más caros que el promedio global

CORRELACIONADA

La subconsulta depende de los resultados de la consulta externa (hace referencia a sus columnas), y se ejecuta para CADA fila de esta.

Orden: ① consulta externa → ② subconsulta por cada fila

```
SELECT e1.nombre, e1.sueldo
    FROM empleados e1
  WHERE sueldo > ( SELECT AVG(salario)
                   FROM empleados e2
                   WHERE e2.deptoID = e1.deptoID );
```

(*) Empleados que ganan más que el promedio de su depto

Subconsultas en la cláusula FROM (DML)

Subconsultas en la cláusula **FROM**: **para realizar una consulta sobre la tabla que se obtiene como resultado de ejecutar otra consulta.**

- La subconsulta **resulta en una tabla (temporal o derivada). Se le debe dar un alias** para poder referenciarla.
- Puede devolver **tantas filas y columnas como sea necesario.**
- Son útiles para simplificar consultas de múltiples pasos.

```
SELECT MAX(promedio.precio_promedio)
  FROM ( SELECT categoria,
                AVG(precio) AS precio_promedio
           FROM producto
          GROUP BY categoria) AS promedio;
```

(*) Máximo precio promedio de categoría

Subconsultas en la cláusula SELECT (DML)

Subconsultas en la cláusula **SELECT**: **para mostrar información agregada relacionada a un registro** devuelto.

- Tienen que ser subconsultas de tipo escalar, que devuelven **una única fila y columna (un solo valor)**.

```
SELECT nombre,
        ( SELECT AVG(salario)
            FROM empleados AS e2
           WHERE e2.depto = e1.depto ) AS salario_prom_dep
  FROM empleados e1;
```

(*) Nombre de cada empleado y salario promedio de su departamento

Subconsultas en INSERT (DML)

El resultado de una **subconsulta** puede usarse **como entrada** de una sentencia **INSERT**.

- Las columnas devueltas por la subconsulta deben coincidir con las con las especificadas en la sentencia INSERT INTO.
- Útil para crear copias de tablas, o para rellenar tablas con datos de otras basándose en criterios específicos.

```
INSERT INTO aprobadosMatem (alumnoID, nombre, nota)
    SELECT a.alumnoID, a.nombre, c.nota
      FROM alumnos a INNER JOIN calificaciones c
        ON (a.alumnoID = c.alumnoID)
    WHERE c.nota > 6;
```

(*) En el uso con INSERT, la subconsulta puede no estar encerrada entre paréntesis.

Subconsultas en UPDATE y DELETE (DML)

Como las sentencias **UPDATE y DELETE** pueden usarse con la cláusula WHERE, estas **pueden utilizar las subconsultas** tal cual se mostraron en las sentencias SELECT.

Ejemplo UPDATE:

```
UPDATE empleados
    SET sueldo = sueldo * 1.1
WHERE departamentoID IN ( SELECT departamentoID
                          FROM departamentos
                          WHERE presupuesto > 80000);
```

(*) Aumenta un 10% el sueldo de empleados de departamentos con presupuesto mayor a 80.000

Ejemplo DELETE:

```
DELETE FROM empleados
WHERE sueldo < (SELECT AVG(sueldo)
                FROM empleados);
```

(*) Borra empleados que ganan menos que el promedio

Operadores de Conjunto (DML)

UNIÓN, INTERSECCIÓN y DIFERENCIA:

sentencia_SELECT

UNION | INTERSECT | EXCEPT [ALL]

sentencia_SELECT

[ORDER BY columna [ASC | DESC]];

- **Requisito: mismo número de columnas**, y **tipo de datos** en las sentencias SELECT.
- **Sin ALL:** elimina duplicados **| Con ALL:** mantiene duplicados.
- **ORDER BY** sólo puede aparecer una vez en la consulta, al final.

UNION : Devuelve las filas del **primer SELECT, más aquellas del segundo**.

```
SELECT ciudad FROM clientes
UNION
SELECT ciudad FROM vendedores;
```

INTERSECT : Devuelve las filas que están **tanto en el primer SELECT como en el segundo**.

```
SELECT ciudad FROM clientes
INTERSECT
SELECT ciudad FROM vendedores;
```

EXCEPT : Devuelve las filas del **primer SELECT que no están en el segundo**.

```
SELECT ciudad FROM clientes
EXCEPT
SELECT ciudad FROM vendedores;
```

(*) No es conmutativa

Vistas (DDL)

Una vista es una **tabla virtual basada en el resultado de una consulta SQL**

Su contenido está definido como una consulta sobre una o varias tablas:

```
CREATE VIEW nombre_vista [ ( nombre_col, ... ) ]
    AS sentencia_SELECT;
```

- Las columnas de la vista se pueden nombrar especificando la lista entre paréntesis. Si no, toma los mismos que los de las columnas de la sentencia SELECT.
- **No almacena datos físicamente**, los datos existen en una o más tablas referenciadas en la consulta. Las vistas **se actualizan dinámicamente**, si los datos de las tablas subyacentes cambian, las vistas que se basan en esas tablas también lo harán.

Vistas (DDL)

Beneficios o propósitos:

- **Seguridad:** permiten ocultar columnas o filas sensibles a ciertos usuarios.
- **Reducción de complejidad y reutilización:** permiten guardar consultas complejas para su reutilización, simplificando el código.
- **Independencia lógica:** si el esquema cambia (se agregan campos, una tabla se divide, etc.), permiten que los usuarios sigan viendo la misma estructura.

```
(*) Vista empleado + jefe
CREATE VIEW v_emp_jef ( empID, empNombre, jefID, jefNombre )
    AS SELECT e.empID, e.nombre, j.empID, j.nombre
        FROM empleados e
         LEFT OUTER JOIN empleados j ON (e.jefeID = j.empID);

(*) Uso como si fuera una tabla
  SELECT v.ventasID, v.fecha, vej.empNombre, vej.jefNombre
    FROM ventas v
   INNER JOIN v_emp_jef vej ON (v.empleadoID = vej.empID)
  WHERE ...;
```

Índices (DDL)

Se utilizan **para acelerar la recuperación de datos** con ciertas condiciones de búsqueda**, basados en uno o más campos**. Evitan escanear toda la tabla fila a fila para encontrar filas (similar al índice de un libro).

```
CREATE INDEX nombre_índice ON tabla (columna1, columna2, …);
```

Índices Agrupados (Clustered)	Determinan el orden físico de los datos de una tabla. **Las filas se almacenan en el disco en el orden** de la/s columna/s indexadas.	**Sólo 1 por tabla** (generalmente la PK). Palabra reservada: CLUSTERED
Índices No Agrupados (Non-Clustered)	No afectan al orden físico de los datos. **Estructura independiente con punteros** a las filas de datos reales.	**Varios por tabla**, para distintos patrones de búsqueda.

Índices Únicos: Refuerzan la integridad impidiendo valores duplicados en la/s columna/s indexadas (UNIQUE).

Índices Compuestos: Son los que abarcan más de una columna simultáneamente.

Índices (DDL)

Los índices permiten acelerar operaciones de búsqueda. Pero **crear índices excesivos puede** bajar el rendimiento de la base de datos. Cada índice requiere **espacio de almacenamiento** y **aumenta el tiempo necesario para insertar, actualizar o eliminar datos** (para el mantenimiento de los índices):

- Indexar las **columnas que se utilizan con frecuencia** en cláusulas **WHERE, JOIN y ORDER BY**.
- Centrarse en las **columnas con alta cardinalidad** (muchos valores distintos). Si una columna tiene pocos valores posibles, un índice sobre esta no aportará ventajas significativas.
- **No** crear índices para tablas pequeñas (con pocos registros).

Ejemplos:

```
CREATE UNIQUE INDEX idx_correo ON empleados (email);

CREATE INDEX idx_nombrecompleto ON empleados (apellido, nombre);
```

Otros Objetos de Bases de Datos (DDL)

Bloques de código SQL pre compilados. Permiten reutilizar ese código en diferentes momentos.

Procedimientos Almacenados (Stored Procedures)

✓ Permite INSERT, UPDATE, DELETE, y devolver SELECT.

✓ Admite parámetros de entrada y de salida.

× No se puede incrustar dentro de una sentencia SQL.

Ejecución: CALL o EXEC nombre_procedimiento.

Uso: Ejecutar lógica de negocio compleja.

```
CREATE PROCEDURE nombre_procedimiento
(param1 tipo_de_datos, param2 tipo_de_datos, ...)
AS
BEGIN
-- Sentencias SQL
END;
```

Funciones definidas por el usuario (UDF)

× No permite modificar datos.

✓ Sólo parámetros de entrada y debe devolver un valor.

✓ Se embebe directamente en SELECT o WHERE.

Ejecución: SELECT función(columna) FROM tabla

Uso: Cálculos repetitivos en expresiones SQL.

```
CREATE FUNCTION nombre_funcion
(param1 tipo_de_datos, param2 tipo_de_datos, ...)
RETURNS tipo_de_datos
AS
RETURN <expresión>
END;
```

Sentencias de Control de Datos (DCL)

Data Control Language (DCL) se utiliza para **controlar permisos y acceso** a la base de datos (seguridad).

Comandos principales: GRANT y **REVOKE** para otorgar y quitar **privilegios** a **usuarios** (o roles) sobre **objetos** de la base de datos.

- **Privilegios (acciones permitidas): SELECT, INSERT, UPDATE, DELETE, EXECUTE**, etc.
- **Objetos**: **tablas**, **vistas**, **procedimientos**, etc.

GRANT – otorgar privilegios

GRANT <privilegios> **ON** <objeto> **TO** <usuario>;

GRANT SELECT, INSERT, DELETE **ON** empleados **TO** usuarioA;

(*) Dar permisos de lectura, inserción y borrado a usuarioA en la tabla empleados

REVOKE – quitar privilegios

REVOKE <privilegios> **ON** <objeto> **FROM** <usuario>;

REVOKE INSERT **ON** empleados **FROM** usuarioA;

(*) Quitar permisos de inserción a usuarioA en la tabla empleados

Sentencias de Control de Transacciones (TCL)

Transaction Control Lenguage (TCL) se utiliza para **administrar las transacciones** en la base de datos.

Una transacción SQL es una secuencia de una o más sentencias (INSERT, UPDATE o DELETE) que se consideran una **unidad de trabajo única e indivisible** (no tiene sentido que se ejecute una sin que se ejecuten las demás).

Los principios **ACID** garantizan que las transacciones sean procesadas de manera confiable:

- **A (Atomicidad):** O bien se completan todos sus pasos, o bien ninguno de ellos.
- **C (Consistencia):** Una transacción debe llevar la base de datos de un estado válido a otro estado válido.
- **I (Aislamiento):** Las transacciones concurrentes no deben interferir entre sí.
- **D (Durabilidad):** Una vez confirmada, sus cambios son permanentes y no se perderán, incluso ante fallos.

Sentencias de Control de Transacciones (TCL)

Sentencias principales del TCL:

- **BEGIN:** Marca el inicio de una transacción. Todas las operaciones posteriores formarán parte de esta transacción.
- **COMMIT:** Guarda de forma permanente los cambios realizados en la transacción. No se pueden deshacer con ROLLBACK.
- **ROLLBACK:** Deshace todas las operaciones desde el último COMMIT o SAVEPOINT. Útil para revertir la base de datos a un estado anterior en el caso de un error.
- **SAVEPOINT** (Opcional): Para tener un ROLLBACK a un punto específico en la transacción en lugar de deshacerla toda. Útil para transacciones muy largas.

```
BEGIN TRANSACTION;  -- Inicia la transacción

-- Operación 1: insertar pedido
INSERT INTO pedidos (id, cliente_id, total)
      VALUES (1, 101, 50.00);

-- Punto de guardado
SAVEPOINT pto_verificacion_1;

-- Operación 2: actualizar inventario
UPDATE inventario
   SET stock = stock - 1
 WHERE producto_id = 101;

-- Si op2 falla revertir sólo hasta el SAVEPOINT
ROLLBACK TO SAVEPOINT pto_verificacion_1;

COMMIT;  -- Finaliza y persiste
```